THE UNIVERSITY OF AZAD JAMMU & KASHMIR,

Department of Software Engineering

LAB 06

Simple Calculator (GUI)

Submitted to: Engr. Dr. Asma Javaid
Submitted By: Jawahir Ali
Session: 2024-28
Roll No: 2024-SE-34
Date: 11-10-2026

Q 1: Simple Calculator (GUI)

Design and develop a simple Calculator with a Graphical User Interface.

Approach

The calculator is written in Python using the Tkinter library. The window follows the sample given in the lab manual: a display on top, and a button grid with digits 0-9, the operators / * - +, the decimal point, CE, C and =. The 0 button spans two columns and the = button spans two rows, as in the sample.

- Display: a read-only Entry widget, right-aligned, showing 0 at start.
- Buttons: created in a loop from a list of (text, row, column, rowspan, columnspan) and placed with the grid layout manager.
- C clears everything (current number and the stored expression).
- CE clears only the number currently being typed.
- Division by zero shows "Error" instead of crashing.
- Keyboard keys (digits, operators, Enter, Backspace, Esc) also work.

Interface (layout preview)



Figure 1: Calculator layout (replace with your own screenshot after running the program)

Source Code (calculator.py)

```
import tkinter as tk


class Calculator:
    def __init__(self, root):
        self.root = root
        self.root.title("Calculator")
        self.root.resizable(False, False)
        self.root.configure(bg="#f3f3f3")

        self.expression = ""      # full expression typed so far
        self.current = "0"        # text shown on the display
```

```
        self.reset_next = False   # True after "=" so next digit starts fresh

        # Display
        self.display = tk.Entry(
            root, font=("Segoe UI", 20), justify="right",
            bd=1, relief="solid", bg="white", state="readonly",
            readonlybackground="white"
        )
        self.display.grid(row=0, column=0, columnspan=5,
                          padx=8, pady=8, ipady=8, sticky="nsew")
        self.update_display()

        # Buttons: (text, row, column, rowspan, columnspan)
        buttons = [
            ("7", 1, 0, 1, 1), ("8", 1, 1, 1, 1), ("9", 1, 2, 1, 1),
            ("/", 1, 3, 1, 1), ("CE", 1, 4, 1, 1),
            ("4", 2, 0, 1, 1), ("5", 2, 1, 1, 1), ("6", 2, 2, 1, 1),
            ("*", 2, 3, 1, 1), ("C", 2, 4, 1, 1),
            ("1", 3, 0, 1, 1), ("2", 3, 1, 1, 1), ("3", 3, 2, 1, 1),
            ("-", 3, 3, 1, 1), ("=", 3, 4, 2, 1),
            ("0", 4, 0, 1, 2), (".", 4, 2, 1, 1), ("+", 4, 3, 1, 1),
        ]

        for text, r, c, rs, cs in buttons:
            btn = tk.Button(
                root, text=text, font=("Segoe UI", 14), width=4, height=2,
                bg="#fbfbfb", activebackground="#e5e5e5", relief="groove",
                command=lambda t=text: self.on_click(t)
            )
            btn.grid(row=r, column=c, rowspan=rs, columnspan=cs,
                     padx=3, pady=3, sticky="nsew")

        for i in range(5):
            root.grid_columnconfigure(i, weight=1)
        for i in range(1, 5):
            root.grid_rowconfigure(i, weight=1)

        # Keyboard support
        root.bind("<Key>", self.on_key)

    # ---------- helpers ----------
    def update_display(self):
        self.display.config(state="normal")
        self.display.delete(0, tk.END)
        self.display.insert(0, self.current)
        self.display.config(state="readonly")

    def on_key(self, event):
        ch = event.char
        if ch in "0123456789.+-*/":
            self.on_click(ch)
        elif ch in ("\r", "="):
            self.on_click("=")
        elif event.keysym == "BackSpace":
            self.on_click("CE")
        elif event.keysym == "Escape":
            self.on_click("C")

    # ---------- main logic ----------
    def on_click(self, key):
        if key.isdigit():
            self.press_digit(key)
        elif key == ".":
            self.press_dot()
        elif key in "+-*/":
```

```
            self.press_operator(key)
        elif key == "=":
            self.calculate()
        elif key == "C":
            self.clear_all()
        elif key == "CE":
            self.clear_entry()
        self.update_display()

    def press_digit(self, d):
        if self.reset_next or self.current == "0" or self.current == "Error":
            self.current = d
            if self.reset_next:
                self.expression = ""
            self.reset_next = False
        else:
            self.current += d

    def press_dot(self):
        if self.reset_next or self.current == "Error":
            self.current = "0."
            self.expression = ""
            self.reset_next = False
        elif "." not in self.current:
            self.current += "."

    def press_operator(self, op):
        if self.current == "Error":
            return
        if self.reset_next:
            # continue calculating with the previous result
            self.expression = ""
            self.reset_next = False
        # add current number + operator to the expression
        self.expression += self.current + op
        self.current = "0"

    def calculate(self):
        if self.current == "Error" or not self.expression:
            return
        full = self.expression + self.current
        try:
            result = eval(full)          # only digits, . and + - * / reach here
            if isinstance(result, float) and result.is_integer():
                result = int(result)
            self.current = str(round(result, 10) if isinstance(result, float) else
result)
        except ZeroDivisionError:
            self.current = "Error"
        except Exception:
            self.current = "Error"
        self.expression = ""
        self.reset_next = True

    def clear_all(self):
        """C : clears everything."""
        self.expression = ""
        self.current = "0"
        self.reset_next = False

    def clear_entry(self):
        """CE : clears only the number currently being typed."""
        self.current = "0"
        if self.reset_next:
            self.expression = ""
            self.reset_next = False
```

```
if __name__ == "__main__":
    root = tk.Tk()
    Calculator(root)
    root.mainloop()
```

How to Run

Save the code as calculator.py and run it with:

```
python calculator.py
```

Test Cases

The calculator logic was checked with the following button sequences:

Button sequence	Expected result	Result
7 + 3 =	10	10
9 - 4 =	5	5
6 * 7 =	42	42
8 / 2 =	4	4
1 / 4 =	0.25	0.25
2.5 + 1.5 =	4	4
12 + 34 =	46	46
7 + 3 = * 2 =	20	20
5 / 0 =	Error	Error
5 + 3 CE 4 =	9	9
5 then C	0	0

Conclusion

A simple calculator GUI was designed and developed using Python Tkinter. It supports addition, subtraction, multiplication, division, decimal numbers, clearing (C and CE) and error handling for division by zero.